
ニューラルネットワーク

活性化関数と多次元配列の計算（3章）

パーセプトロン → ニューラルネットワーク

この章の流れ

パーセプトロンからニューラルネットワークへ

活性化関数

多次元配列の計算

パーセプトロンからニューラルネットワークへ

この章のゴール

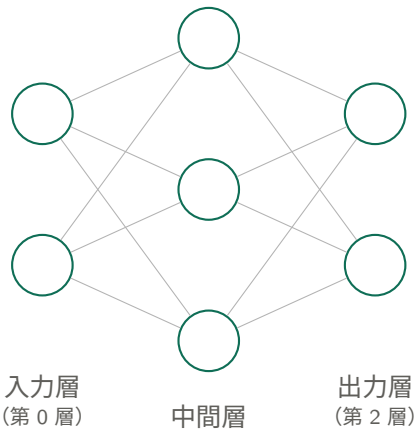
- パーセプトロンとニューラルネットワークの **共通点と相違点**を理解する
- **活性化関数**（ステップ・シグモイド・ReLU）の意味と実装を知る
- **行列の積**でニューラルネットワークの計算を表せるようになる

キーアイデア

パーセプトロンは「重みを人が決める」。

ニューラルネットワークは「重みをデータから自動で学習する」。

ニューラルネットワークの構造



- 左から **入力層** → **中間層** → **出力層**
- 中間層は「隠れ層」とも呼ぶ
- 第 0 層・第 1 層・第 2 層 という数え方は実装の添字と一致して便利
- 信号が流れる各点を **ニューロン (ノード)** と呼ぶ

パーセプトロンの復習

入力 x_1, x_2 を受け取り、出力 y を返すパーセプトロン：

$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases} \quad (3.1)$$

- w_1, w_2 ：重み — 各信号の重要度をコントロール
- b ：バイアス — ニューロンの発火のしやすさをコントロール

総和が 0 を超えれば 1、そうでなければ 0 を出力する。

関数を使って書き直す

式 (3.1) は冗長なので、1 つの関数 $h(x)$ を導入する。

$$y = h(b + w_1x_1 + w_2x_2) \quad (3.2)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases} \quad (3.3)$$

活性化関数 (activation function)

総和を受け取り、出力へ変換する関数 $h(x)$ 。

ここではステップ関数だが、これを別の関数に置き換えるのが次のテーマ。

活性化関数の登場 — 2 段階に分ける

式 (3.2) の計算は、2 段階に分けると見通しが良い。

$$a = b + w_1x_1 + w_2x_2 \quad (3.4)$$

$$y = h(a) \quad (3.5)$$

1. まず **総和** a を計算する (式 3.4)
2. 次に活性化関数 $h()$ に通して **出力** y を得る (式 3.5)

$$x_1, x_2 \longrightarrow \boxed{a = b + \sum w_i x_i} \longrightarrow \boxed{y = h(a)} \longrightarrow y$$



活性化関数

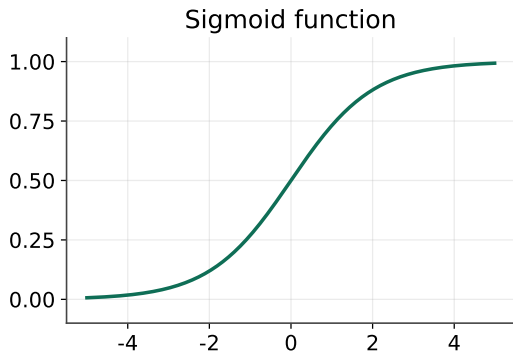
シグモイド関数

ニューラルネットワークでよく使う活性化関数：

$$h(x) = \frac{1}{1 + \exp(-x)} \quad (3.6)$$

$\exp(-x) = e^{-x}$, $e \approx 2.718$ (ネイピア数)。

- $h(1.0) = 0.731 \dots$
- $h(2.0) = 0.880 \dots$
- 連続的な実数を出力する



ステップ関数の実装

まずは素直な実装（スカラーのみ対応）：

```
def step_function(x):  
    if x > 0:  
        return 1  
    else:  
        return 0
```

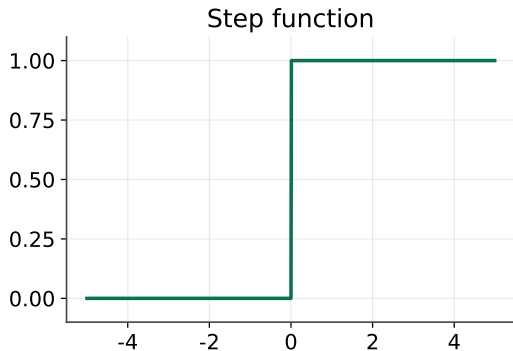
NumPy 配列にも対応させる（bool 配列を整数に変換）：

```
def step_function(x):  
    return np.array(x > 0, dtype=np.int_)  
#  $x > 0$  が True/False の配列になり、astype/int変換で 1/0 になる
```

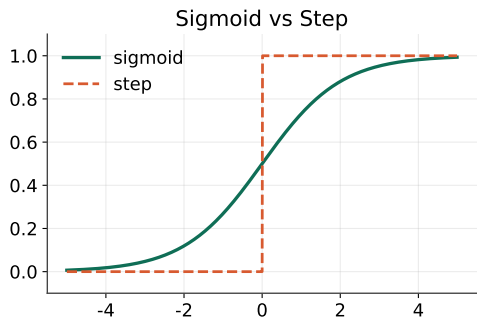
ステップ関数のグラフ

- 0 を境に 0 から 1 へ階段状に切り替わる
- 出力は 0 か 1 の 2 値のみ

$x = \text{np.arange}(-5.0, 5.0, 0.1)$ を入力し、出力をプロットしたもの。



シグモイド関数とステップ関数の比較



相違点

- **滑らかさ**：シグモイドは連続的な曲線／ステップは階段状
- **出力の値**：ステップは0/1の2値／シグモイドは連続値

共通点

- 入力が小さいと0に近く、大きいと1に近づく
- 出力は必ず0～1の間

「滑らかさ」が、後の **学習** で重要になる。

非線形関数 — なぜ必要か

活性化関数には非線形関数を使わなければならない。

- 線形関数： $h(x) = cx$ （グラフが1本の直線、 c は定数）
- 非線形関数：直線では表せない関数

線形だと層を深くする意味が消える

線形 $h(x) = cx$ を3層重ねると

$$y = h(h(h(x))) = c \cdot c \cdot c \cdot x = c^3 x = a x$$

結局 $y = ax$ 、つまり1層と同じになってしまう。

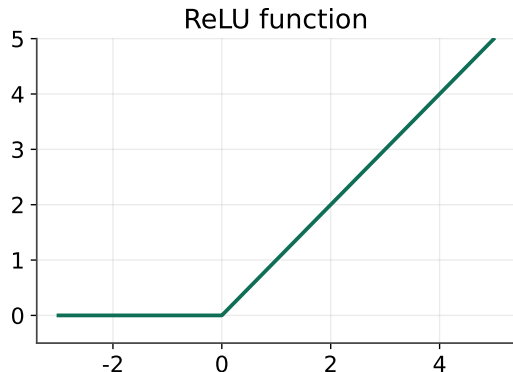
だから層を重ねる利点を活かすには、非線形関数が必要。

ReLU 関数

近年の主流となっている活性化関数：

$$h(x) = \begin{cases} x & (x > 0) \\ 0 & (x \leq 0) \end{cases} \quad (3.7)$$

- 正ならそのまま、0 以下なら 0
- `np.maximum(0, x)` の一行で実装





多次元配列の計算

多次元配列

「数字の集合」。np.ndim() で次元数、.shape で形状を取得。

```
A = np.array([1, 2, 3, 4])
np.ndim(A) # -> 1
A.shape # -> (4,) 1次元配列

B = np.array([[1,2], [3,4], [5,6]])
np.ndim(B) # -> 2
B.shape # -> (3, 2) 3行2列の行列
```

横の並びを 行 (row)、縦の並びを 列 (column) と呼ぶ。

行列の積

「左の行」 × 「右の列」を要素ごとに掛けて足す。

```
A = np.array([[1,2], [3,4]])  
B = np.array([[5,6], [7,8]])  
np.dot(A, B) # 行列の積 (A @ B でも同じ)  
# array([[19, 22],  
#        [43, 50]])
```

注意：順序で結果が変わる

一般に $AB \neq BA$ 。

形状の一致ルール

対応する次元の要素数を一致させる

$$\underbrace{A}_{m \times n} \cdot \underbrace{B}_{n \times p} = \underbrace{C}_{m \times p}$$

- 左の列数と右の行数（内側の n ）を一致させる
- 結果は外側の $m \times p$
- 一致しないとエラーになる

$$\text{例：} 2 \times 3 \cdot 3 \times 4 \rightarrow 2 \times 4$$

ニューラルネットワークの行列の積

バイアスと活性化関数を省いた、重みだけの単純なネットワーク：

```
X = np.array([1, 2]) # 入力 shape (2,)  
W = np.array([[1, 3, 5],  
              [2, 4, 6]]) # 重み shape (2, 3)  
Y = np.dot(X, W) # shape (3,)  
# Y = [5, 11, 17]
```

- X の要素数 (2) と W の行数 (2) を一致させる
- 出力ニューロン 3 つ分を **for を使わず一度に**計算できる

1 層分の順伝播

バイアスと活性化関数を戻すと、1 層分の計算になる。

$$A = X \cdot W + B \quad Z = h(A)$$

1. $A = X \cdot W + B$ ：重み付き総和とバイアス（式 3.4 相当）
2. $Z = h(A)$ ：活性化関数に通す（式 3.5 相当）

$$X \rightarrow \boxed{A = XW + B} \rightarrow \boxed{Z = h(A)} \rightarrow Z$$

まとめ

- パーセプトロンとニューラルネットワークの主な違いは **活性化関数**
 - パーセプトロン：ステップ関数
 - ニューラルネットワーク：シグモイドや ReLU などの滑らかな／非線形な関数
- 活性化関数に **非線形関数**を使うことで、層を深くする意味が生まれる
- ニューラルネットワークの計算は **行列の積** (`np.dot`) で効率よく表せる
- 行列の積では **対応する次元の要素数を一致させる**ことが鉄則

次章：これらを組み合わせて、多層ネットワーク全体の順伝播を実装する。